Discovery Executive Summary

Project: test-discovery-15july · **Generated:** 15/07/2026, 14:53:10

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes and Business Logic in Components with oversized frontend components violating SRP.

EXECUTIVE SUMMARY

This multi-agent web application demonstrates a well-established service layer pattern but suffers from significant architectural hotspots. The system shows excellent separation of concerns in the backend with 58 service classes and 12 controllers, but contains several god-class anti-patterns, with the largest service file reaching 781 LOC. Frontend architecture exhibits critical issues with oversized components (3220 LOC store, 1486 LOC detail component) and business logic scattered across view components rather than dedicated services. The overall architecture violates single responsibility principles and presents change amplification risks as the system scales.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	146	GOOD

H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	0	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	15	MODERATE
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	1	MODERATE
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	0	GOOD
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	5%	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	209	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	18	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	7	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
F3. God / Oversized Components	Split useAppStore (3220 LOC) into AuthContext, WorkflowContext, AgentExecutionContext. Break AgentDetail (1486 LOC) into focused components.	HIGH RISK	CRITICAL
H3. Missing Repository Pattern	Introduce repository interfaces for User, Team, Integration, RepoAst domains with dependency injection. Consolidate 15 direct DB access points.	MODERATE	HIGH
H7. God Classes	Split repoAstService (781 LOC) into RepoAnalysisService, AstParsingService, SymbolIndexService, RepoCacheService using composition pattern.	MODERATE	MEDIUM

F1. Business Logic in Components	Extract workflow logic into AgentWorkflowService, PublishService. Create custom hooks for clean component interfaces.	MODERATE	MEDIUM
F4. Prop Drilling / Global State Abuse	Introduce focused contexts (SetupContext, AgentDetailContext). Split monolithic useAppStore to reduce re-render scope.	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Improved Maintainability**: Breaking god components and classes into focused units reduces change amplification and enables independent testing of business logic
- **Enhanced Testability**: Repository pattern and service extraction enable isolated unit testing without database dependencies or complex UI setup
- **Better Team Productivity**: Smaller, focused components reduce merge conflicts and allow multiple developers to work on different features simultaneously
- **Reduced Technical Debt**: Clear separation of concerns and consistent architectural patterns make the codebase more approachable for new team members
- **Future-Ready Architecture**: Proper domain boundaries and dependency injection prepare the system for microservices extraction and technology migrations

The complete Architecture & Design Hotspots Analysis has been saved to `docs/discovery/01-architecture-design.md`. The analysis identified critical frontend architectural issues with oversized components and moderate backend concerns around repository patterns and god classes, resulting in an overall High Risk rating that requires immediate attention to prevent further technical debt accumulation.